Build a workflow in Microsoft Foundry

In this exercise, you'll use the Microsoft Foundry portal to create a workflow. Workflows are UI-based tools that allow you to define sequences of actions involving AI agents. For this exercise, you'll create a workflow that helps resolve customer support requests.

Workflow overview

- Collect incoming support tickets

  The workflow starts with a predefined array of customer support issues. Each item in the array represents an individual support ticket submitted to ContosoPay.

- Process tickets one at a time

  A for-each loop iterates over the array, ensuring each support ticket is handled independently while using the same workflow logic.

- Classify each ticket with an AI agent

  For each ticket, the workflow invokes a Triage Agent to classify the issue as Billing, Technical, or General, along with a confidence score.

- Handle uncertainty with conditional logic

  If the confidence score is below a defined threshold, the workflow recommends additional info for that ticket.

- Route based on issue category

  Billing issues are flagged for escalation and removed from the automated resolution path. Technical and General issues continue through automated handling.

- Generate a recommended response

  For non-billing tickets, the workflow invokes a Resolution Agent to draft a category-appropriate support response.

This exercise should take approximately **30** minutes to complete.

> **Note**: The workflow builder in Microsoft Foundry is currently in preview. You may experience some unexpected behavior, warnings, or errors. If you encounter any issues that block your progress, you may need to start over with a new project and workflow.

Prerequisites

Before starting this exercise, ensure you have:

- Visual Studio Code installed on your local machine
- An active Azure subscription
- Python 3.13 installed
- Git installed on your local machine

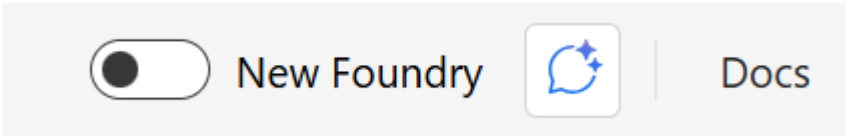
> * Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

Create a Foundry project

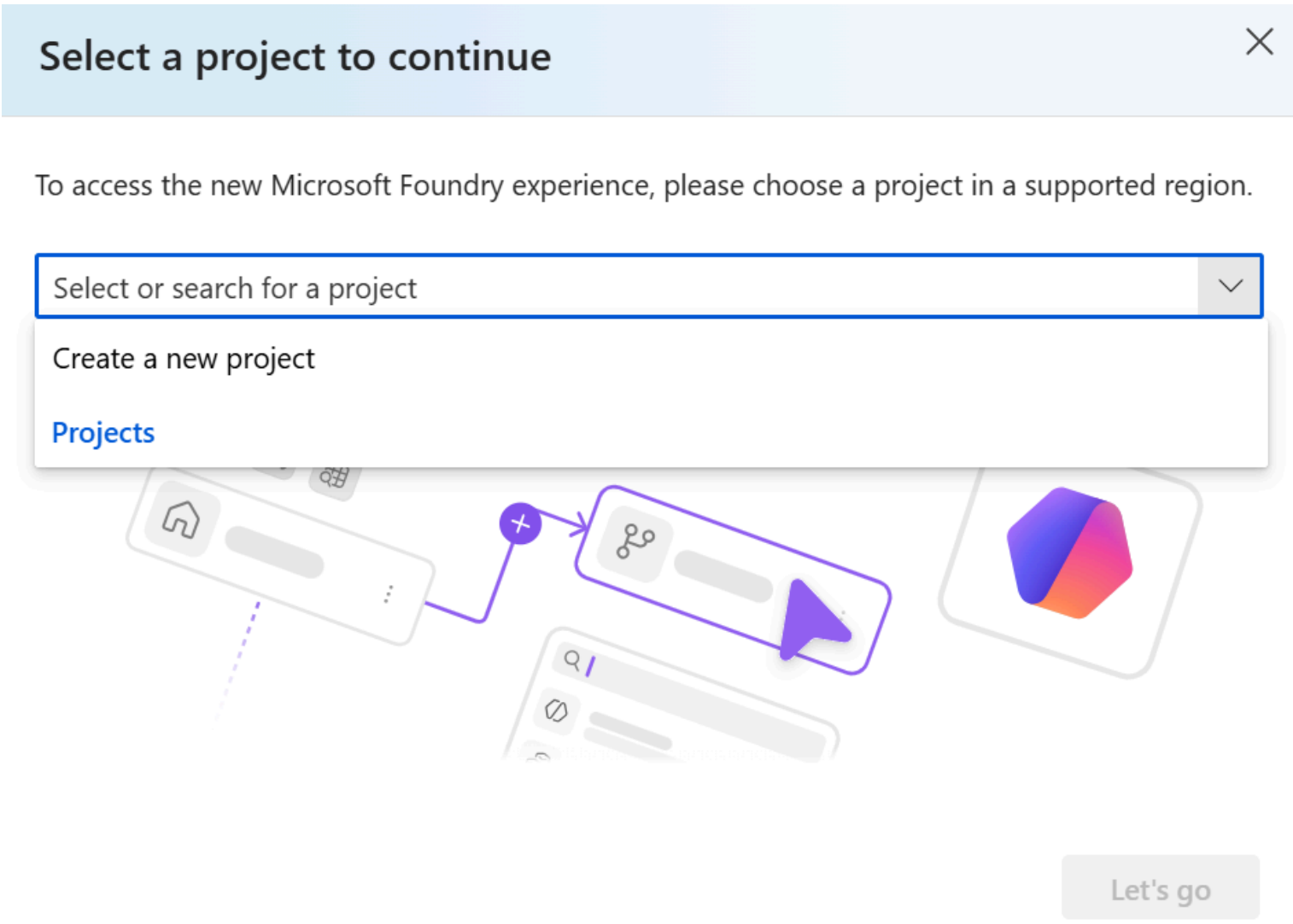
Let's start by creating a Foundry project.

1. In a web browser, open the Foundry portal at `https://ai.azure.com` and sign in using your Azure credentials.

2. Ensure the **New Foundry** toggle is set to *On*.



3. You may be prompted to create a new project before continuing to the New Foundry experience. Select **Create a new project**.



   If you're not prompted, select the projects drop down menu on the upper left, and then select **Create new project**.

4. Enter a name for your Foundry project in the textbox and select **Create**.

   Wait a few moments for the project to be created. The new Foundry portal home page should appear with your project selected.

5. Close the **Welcome to the new Microsoft Foundry** dialog if it appears.

   The dialog may prompt you to create an agent which is not necessary at this time. Agents will be created in a later step.

Create a customer support triage workflow

In this section, you'll create a workflow that helps triage and respond to customer support requests for a fictional company called ContosoPay. The workflow uses two AI agents that classify and respond to support tickets.

1. On the Foundry portal home page, select **Build** from the toolbar menu.

2. On the left-hand menu, select **Agents** then select the **Workflows** tab.

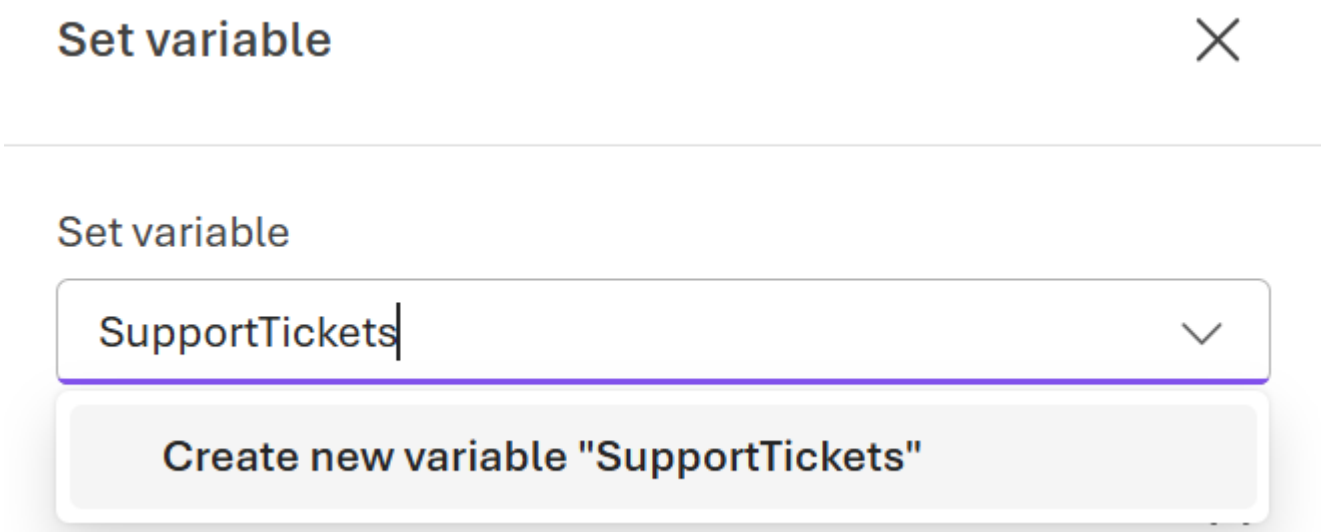
3. In the upper right corner, select **Create** > **Blank workflow** to create a new blank workflow.

   The type of workflow you'll create in this exercise is a sequential workflow. However, starting with a blank workflow will simplify the process of adding the necessary nodes.

4. Select **Save** in the visualizer to save your new workflow. In the dialog box, enter a name for your workflow, such as *ContosoPay-Customer-Support-Triage*, and then select **Save**.

Create a ticket array variable

1. In the workflow visualizer, select the **+** (plus) icon to add a new node.
2. In the workflow actions menu, under **Data transformation**, select **Set variable** to add a node that initializes an array of support tickets.
3. In the **Set variable** node editor, enter a name for a new variable, such as *SupportTickets*.



   The new variable should appear as `Local.SupportTickets`.

4. In the **To value** field, enter the following array that contains sample support tickets:

```
[
 "The API returns a 403 error when creating invoices, but our API key hasn't changed.",
 "Is there a way to export all invoices as a CSV?",
 "I was charged twice for the same invoice last Friday and my customer is also seeing two receipts. Can someone fix this?"]
```

5. Select **Done** to save the node.

Add a for-each loop to process tickets

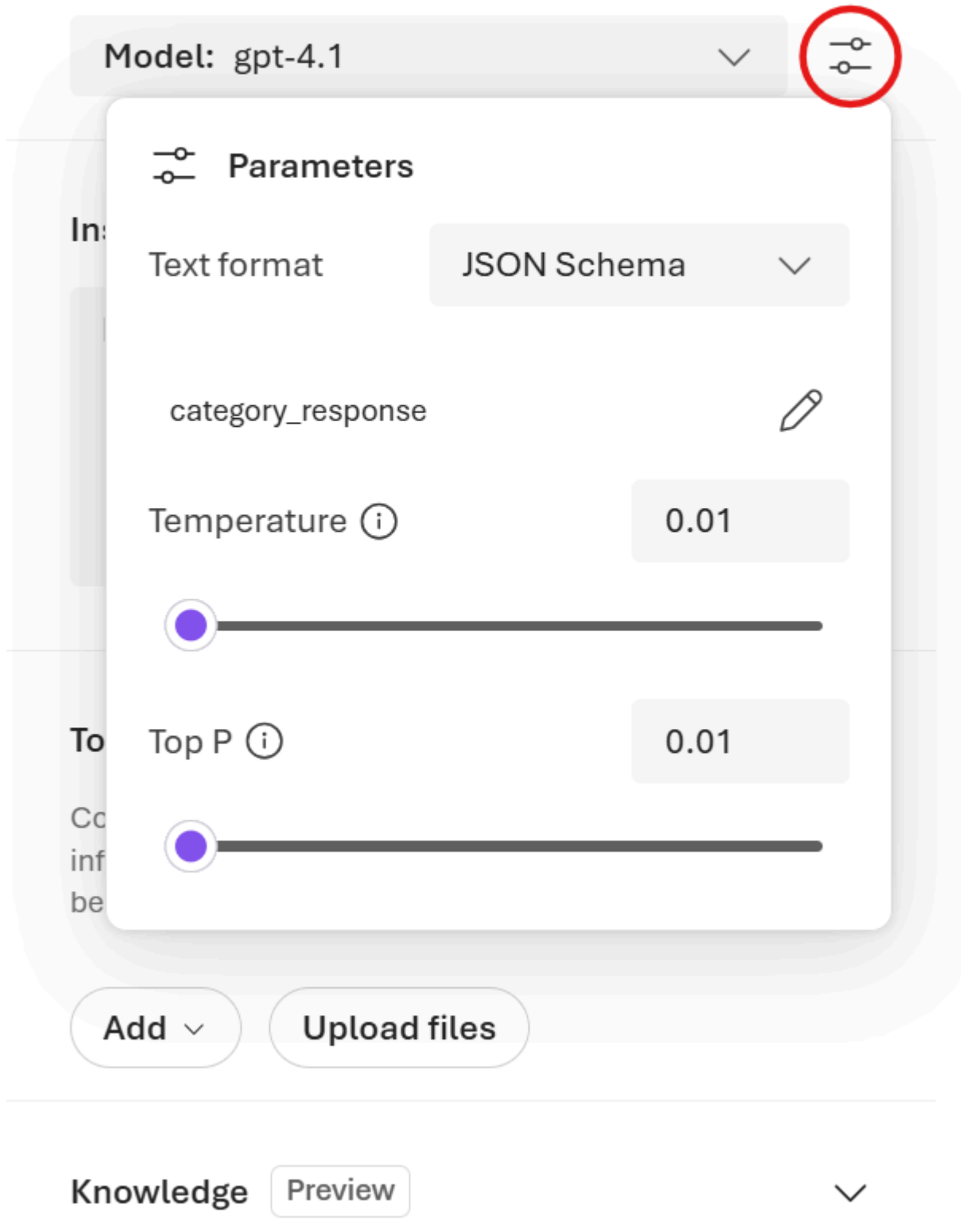
1. Select the **+** (plus) icon below the **Set variable** and create a **For each** node to process each support ticket in the array.
2. In the **For each** node editor, set the **Select the items to loop for each** field to the variable you created earlier: `Local.SupportTickets`.
3. In the **Loop Value Variable** field, create a new variable named `CurrentTicket`.
4. Select **Done** to save the node.

Invoke an agent to classify the ticket

1. Select the **+** (plus) icon within the **For each** node to add a new node that classifies the current support ticket.
2. In the workflow actions menu, under **Invoke**, select **Agent** to add an agent node.
3. In the **Agent** node editor, under **Select an agent**, select **Create new agent**.
4. Enter an agent name such as *Triage-Agent* and select **Create**.

Configure the agent settings

1. In the editor, under **Details**, select the **Parameters** button near the model name.



2. In the **Parameters** pane, next to **Text format**, select **JSON Schema**.

3. In the **Add response format** pane, enter the following definition and select **Save**:

```
Code
```
Copy

```
{
"name": "category_response",
"schema": {
    "type": "object",
    "properties": {
        "customer_issue": {
            "type": "string"
        },
        "category": {
            "type": "string"
        },
        "confidence": {
            "type": "number"
        }
    },
    "additionalProperties": false,
    "required": [
        "customer_issue",
        "category",
        "confidence"
    ]
},
"strict": true
}
```

4. In the Agent Details pane, set the **Instructions** field to the following prompt:

Code Copy

```
Classify the user's problem description into exactly ONE category from the list below.
Provide a confidence score from 0 to 1.

Billing
- Charges, refunds, duplicate payments
- Missing or incorrect payouts
- Subscription pricing or invoices being charged

Technical
- API errors, integrations, webhooks
- Platform bugs or unexpected behavior

General
- How-to questions
- Feature availability
- Data exports, reports, or UI navigation

Important rules
- Questions about exporting, viewing, or downloading invoices are General, not Billing
- Billing ONLY applies when money was charged, refunded, or paid incorrectly
```

5. Select **Node settings** to configure the input and output of the agent.
6. Set the **Input message** field to the `Local.CurrentTicket` variable.
7. Under **Save agent output message as**, create a new variable named `TriageOutputText`.
8. Under **Save the output json_object as**, create a new variable named `TriageOutputJson`.

9. Select **Done** to save the node.

Handle low-confidence classifications

1. Select the **+** (plus) icon below the **Invoke agent** node to add a new node that handles low-confidence classifications.
2. In the workflow actions menu, under **Flow**, select **If/Else** to add a conditional logic node.
3. In the **If/Else** node editor, select the **Add a path** button to create the if-branch condition, then select the pencil icon to edit the condition.
4. Set the **Condition** field to the following expression to check if the confidence score is above 0.6:

```
Local.TriageOutputJson.confidence > 0.6
```

5. Select **Done** to save the node.

Recommend additional info for low-confidence tickets

1. In the visualizer, under the **Else** branch of the **If/Else condition** node, select the **+** (plus) icon to add a new node that recommends additional information for low-confidence tickets.
2. In the workflow actions menu, under **Basics**, select **Deliver a message** to add a send message activity.
3. In the **Deliver a message** node editor, set the **Message to send** field to the following response:

```
The support ticket classification has low confidence. Requesting more details about the
issue: "{Local.CurrentTicket}"
```

4. Select **Done** to save the node.

Route the ticket based on category

In this section, you'll add conditional logic to route the ticket based on its classified category if the confidence score is high enough.

1. In the visualizer, under the **If** branch of the **If/Else condition** node, select the **+** (plus) icon to add a new node that routes the ticket based on its category.
2. In the workflow actions menu, under **Flow**, select **If/Else** to add another conditional logic node.
3. In the **If/Else** node editor, select the **Add a path** button to create the if-branch condition, then select the pencil icon to edit the condition.
4. Set the **If Condition** to the following expression to check if the ticket category is "Billing":

```
Local.TriageOutputJson.category = "Billing"
```

5. Select the **+** (plus) icon under the **If** branch of the **If/Else** node to add a new node that drafts a response for non-billing tickets.
6. In the workflow actions menu, under **Basics**, select **Deliver a message** to add a send message activity.
7. In the **Deliver a message** node editor, set the **Message to send** to the following response:

Code Copy

```
Escalate billing issue to human support team.
```

8. Select **Done** to save the node.

Generate a recommended response

1. In the visualizer, select the **+** (plus) icon under the **Else** branch of the second **If/Else** node to add a new node that drafts a response for non-billing tickets.
2. In the workflow actions menu, under **Invoke**, select **Agent** to add an agent node.
3. In the **Agent** node editor, select **Create new agent**.
4. Enter an agent name such as *Resolution-Agent* and select **Create**.
5. In the agent editor, set the **Instructions** field to the following prompt:

Code Copy

```
You are a customer support resolution assistant for ContosoPay, a B2B payments and invoicing platform.

Your task is to draft a clear, professional, and friendly support response based on the issue category and customer message.

Guidelines:
If the issue category is Technical:
Suggest 1–2 common troubleshooting steps at a high level.

Avoid asking for logs, credentials, or sensitive data.

Do not imply fault by the customer.
If the issue category is General:
Provide a concise, helpful explanation or guidance.
Keep the response under 5 sentences.

Tone:
Professional, calm, and supportive
Clear and concise
No emojis

Output:
Return only the drafted response text.
Do not include internal reasoning or analysis.
```

6. Select **Node settings** to configure the input and output of the agent.
7. Set the **Input message** field to the `Local.TriageOutputText` variable.
8. Under **Save agent output message as**, create a new variable named `ResolutionOutputText`.
9. Select **Done** to save the node.

Preview the workflow

1. Select the **Save** button to save all changes to your workflow.
2. Select the **Preview** button to start the workflow.

3. In the chat window that appears, enter some text to trigger the workflow, such as `Start processing support tickets.`

4. Observe the workflow as it processes each support ticket in sequence. Review the messages generated by the workflow in the chat window.

   You should see some output indicating that billing issues are being escalated, while technical and general issues receive drafted responses. For example:

```
Current Ticket:
The API returns a 403 error when creating invoices, but our API key hasn't changed.



Copilot said:
Thank you for reaching out about the 403 error when creating invoices. This error typically
indicates a permissions or access issue.
Please ensure that your API key has the necessary permissions for invoice creation and that
your request is being sent to the correct endpoint.
If the issue persists, try regenerating your API key and updating it in your integration to
see if that resolves the problem.
```

Use your workflow in a client application

Now that you've built and tested your workflow in the Foundry portal, you can also invoke it from your own code using the Azure AI Projects SDK. This allows you to integrate the workflow into your applications or automate its execution.

Clone the starter code repository

For this exercise, you'll use starter code that will help you connect to your Foundry project and invoke a workflow.

1. In VS Code, open the Command Palette (**Ctrl+Shift+P** or **View > Command Palette**).

2. Type **Git: Clone** and select it from the list.

3. Enter the repository URL:

```
https://github.com/MicrosoftLearning/mslearn-ai-agents.git
```

4. Choose a location on your local machine to clone the repository.

5. When prompted, select **Open** to open the cloned repository in VS Code.

6. Once the repository opens, select **File > Open Folder** and navigate to `mslearn-ai-agents/Labfiles/06-build-workflow-ms-foundry`, then choose **Select Folder**.

7. In the Explorer pane, expand the **Python** folder to view the code files for this exercise.

Configure the application

1. In the browser, return to the workflow visualizer in the Foundry portal.

2. Select **Code** in the upper right corner of the visualizer. Then select **.env variables** to view the environment variables required to connect to your Foundry project from code.

3. Copy the value of the **AZURE_EXISTING_AIPROJECT_ENDPOINT** variable, which is the endpoint URL for your Foundry project. You'll need this value to connect to your project in VS Code.

4. In VS Code, right-click on the **requirements.txt** file and select **Open in Integrated Terminal**.

5. In the terminal, enter the following command to install the required Python packages in a virtual environment:

Code Copy

```
python -m venv labenv
.\labenv\Scripts\Activate.ps1
pip install -r requirements.txt
```

6. Open the **.env** file, replace the **your_project_endpoint** placeholder with the endpoint for your project (copied from the code tab of the workflow visualizer). Use **Ctrl+S** to save the file after making these changes.

Invoke the workflow from code

Now you're ready to create a project that invokes a workflow. Let's get started!

1. Open the **workflow.py** file in the code editor.

2. Review the code in the file, noting that it contains strings for each agent name and instructions.

3. Find the comment **Add references** and add the following code to import the classes you'll need:

Code Copy

```
# Add references
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
```

4. Find the comment **Connect to the agents client**, and add the following code to create an AgentsClient connected to your project:

Code Copy

```
# Connect to the AI Project client
with (
    DefaultAzureCredential() as credential,
    AIProjectClient(endpoint=endpoint, credential=credential) as project_client,
    project_client.get_openai_client() as openai_client,
):
```

Now you'll add code that uses the AgentsClient to create multiple agents, each with a specific role to play in processing a support ticket.

> **Tip**: When adding subsequent code, be sure to maintain the right level of indentation.

5. Find the comment **Specify the workflow** and add the following code:

Code Copy

```
# Specify the workflow
workflow = {
    "name": "ContosoPay-Customer-Support-Triage"
}
```

Be sure to use the name and version of the workflow you created in the Foundry portal.

6. Find the comment **Create a conversation and run the workflow**, and add the following code to create a conversation and invoke your workflow:

Code Copy

```
# Create a conversation and run the workflow
conversation = openai_client.conversations.create()
print(f"Created conversation (id: {conversation.id})")

stream = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference" : {"name" : workflow["name"], "type": "agent_reference"}},
    input="Start",
    stream=True,
)
```

This code will stream the output of the workflow execution to the console, allowing you to see the flow of messages as the workflow processes each ticket.

7. Find the comment **Process events from the workflow run**, and add the following code to process the streamed output and print messages to the console:

Code Copy

```
# Process events from the workflow run
for event in stream:
    if (event.type == "response.completed"):
        print("\nResponse completed:")
        response = openai_client.responses.retrieve(event.response.id)
        print_workflow_output(response.output_text)
```

This code listens for the completion of the workflow response and then retrieves and prints the final output text to the console. The `print_workflow_output` function is a helper function defined in the code file that formats the output for easier reading.

8. Find the comment **Clean up resources**, and enter the following code to delete the conversation when it is no longer required:

Code Copy

```
# Clean up resources
openai_client.conversations.delete(conversation_id=conversation.id)
print("\nConversation deleted")
```

9. Use the **CTRL+S** command to save your changes to the code file.

Test the client application

Now you're ready to run your code and watch your AI agents collaborate.

Prerequisites

Create a Foundry project

Create a customer support triage workflow

Create a ticket array variable

Add a for-each loop to process tickets

Invoke an agent to classify the ticket

Handle low-confidence classifications

Recommend additional info for low-confidence tickets

Route the ticket based on category

Generate a recommended response

Preview the workflow

Use your workflow in a client application

Test the client application

Clean up

1. In the integrated terminal, run the following commands:

```
az login
```

```
python workflow.py
```

2. Wait a moment for the workflow to process the tickets. As the workflow runs, you should see output in the console indicating the progress of the workflow, including messages generated by the agents and status updates for each action in the workflow.
3. When the workflow completes, you should see some output similar to the following:

```
Response completed:
Current Ticket:
The API returns a 403 error when creating invoices, but our API key hasn't changed.
{"customer_issue":"API returns a 403 error when creating invoices, API key
unchanged.","category":"Technical","confidence":1}Thank you for contacting us about the 403
error when creating invoices with the API. This error typically relates to permission issues.
Please ensure your API key has the necessary permissions for invoice creation and that the
endpoint URL is correct. If the issue persists, try regenerating the API key and updating it
in your application.
...
```

In the output, you can see how the workflow completes each support ticket, including the classification of each ticket and the recommended response or escalation. Great work!

4. When you're finished, enter `deactivate` in the terminal to exit the Python virtual environment.

Clean up

If you've finished exploring workflows in Microsoft Foundry, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. Navigate to the Azure portal at `https://portal.azure.com` and view the contents of the resource group where your Foundry project was deployed.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.